

Cloud Threat Intelligence Dashboard

Comprehensive Codex Build Plan, Secure Architecture, and Deployment Guide

1. Executive Summary

Build a hosted cybersecurity dashboard that accepts an indicator of compromise (IOC), such as an IP address, domain, URL, or file hash, then enriches it using multiple threat intelligence and internet-exposure APIs. The system normalizes the results, calculates a transparent risk score, generates an analyst-style explanation, and displays the evidence in a clean dashboard.

Primary goal: ship a secure, deployable, resume-ready project in one month or less using Codex for implementation assistance while keeping the architecture maintainable and safe.

Important scope boundary: this is a passive threat intelligence and enrichment tool, not an active exploitation scanner, IDS, firewall, or DDoS testing tool.

2. Final Product Outcome

Area	Target Outcome
User workflow	User enters an IP, domain, URL, or hash, clicks Analyze, and receives a risk report.
Threat sources	VirusTotal, AbuseIPDB, AlienVault OTX, Shodan, Censys, URLScan, and IPinfo.
Output	Risk score, source-specific evidence, IOC details, exposed service context, geolocation/ASN context, and AI-generated analyst summary.
Deployment	Frontend hosted on Vercel or Cloud Run static hosting. Backend hosted on Google Cloud Run.
Security posture	API keys hidden server-side, validated inputs, rate limits, safe database access, logging, caching, and abuse prevention.
Portfolio value	Demonstrates cybersecurity, cloud deployment, API orchestration, secure backend design, data normalization, and dashboard development.

3. Simple End-to-End Flow

1. User opens the dashboard and submits an IOC.
2. Frontend sends a POST request to /api/analyze with the IOC string.
3. Backend validates and classifies the IOC as IP, domain, URL, or hash.
4. Backend chooses the appropriate connectors based on IOC type.
5. Backend calls external APIs asynchronously with server-side API keys.
6. Backend normalizes all tool results into one internal schema.
7. Risk engine calculates score and severity.
8. Optional AI summary converts technical evidence into analyst-readable language.
9. Result is cached and saved to history.
10. Frontend renders dashboard cards, charts, evidence tables, and report export.

4. Tool Integration Matrix

Tool	Use Case	Inputs	Output Used in Dashboard
------	----------	--------	--------------------------

VirusTotal	Reputation lookup across security vendors	IP, domain, URL, hash	Detections, reputation, categories, related files/URLs when available.
AbuseIPDB	IP abuse reputation	IP	Abuse confidence score, report count, usage type, ISP, domain, country.
AlienVault OTX	Threat intelligence pulses and IOC context	IP, domain, URL, hash	Pulse count, malware families, tags, references, related indicators.
Shodan	Internet exposure and service discovery	IP, domain	Open ports, banners, products, vulnerabilities if available.
Censys	Structured host/certificate/service exposure	IP, domain	Services, protocols, TLS/certificate context, autonomous system data.
URLScan	URL behavior and page analysis	URL, domain	Scan verdict, screenshot link, redirects, page requests, domains contacted.
IPinfo	IP ownership and geolocation context	IP	ASN, organization, city/region/country, hostname, privacy flags if available.

5. Recommended Tech Stack

Layer	Technology	Reason
Frontend	React + TypeScript + Tailwind	Fast dashboard development, type safety, clean UI.
Backend	FastAPI + Python	Excellent async API orchestration and validation with Pydantic.
Database	PostgreSQL with SQLAlchemy ORM, or Firestore	Stores cached lookups and scan history. ORM prevents raw SQL injection patterns.
Cache	Redis optional, or database-level cache table	Reduces API cost/rate-limit pressure and improves speed.
AI summary	Gemini, OpenAI, or Claude optional	Generates readable analyst summaries from normalized evidence.
Deployment	Docker + Google Cloud Run	Secure, scalable, serverless container deployment.
Secrets	Google Secret Manager	Keeps API keys out of code, frontend, and GitHub.
Charts	Recharts	Simple risk charts and source breakdowns.
Auth	Optional Clerk/Auth.js later	Not required for MVP, useful if adding user accounts.

6. Architecture

Preferred architecture: frontend and backend are separate. The frontend never calls security APIs directly. The backend is the only place where API keys exist.

User Browser

- > React Dashboard
- > FastAPI Backend on Cloud Run
- > Input Validator and IOC Classifier
- > Async Tool Connectors
 - > VirusTotal
 - > AbuseIPDB
 - > AlienVault OTX

- > Shodan
- > Censys
- > URLScan
- > IPinfo
- > Normalization Layer
- > Risk Scoring Engine
- > AI Summary Engine
- > PostgreSQL/Firestore Cache and History
- > JSON Response to Frontend

7. Core Data Model

Entity	Important Fields
AnalysisRequest	id, input_value, input_type, normalized_value, user_ip_hash, created_at, status.
ToolResult	id, analysis_id, source_name, status, raw_json, normalized_json, error_message, created_at.
RiskReport	id, analysis_id, score, severity, summary, recommended_actions, created_at.
CacheEntry	cache_key, input_type, input_value, source_name, response_json, expires_at.
AuditLog	id, event_type, request_id, source_ip_hash, metadata_json, created_at.

8. API Endpoint Design

Endpoint	Method	Purpose	Security Notes
/api/health	GET	Health check for deployment.	No sensitive data returned.
/api/analyze	POST	Submit IOC for analysis.	Strict validation, rate limit, request size limit, no raw SQL.
/api/results/{analysis_id}	GET	Fetch completed analysis.	Validate UUID, return only safe fields.
/api/history	GET	Recent public or local history, depending on scope.	Do not expose raw user identifiers or secrets.
/api/report/{analysis_id}/export	GET	Export report as JSON or PDF.	Validate ID and sanitize generated content.
/api/sources/status	GET	Show enabled connectors and degraded sources.	Do not reveal API keys or exact secret names.

9. Input Validation and SQL Injection Prevention

Codex must implement input validation before any analysis, database query, cache lookup, or external API call. Invalid input should return HTTP 400 with a generic message.

- Use Pydantic request models with strict field lengths and types.
- Classify IOC type using safe parsing libraries: ipaddress for IPs, urllib.parse for URLs, regex with length limits for hashes and domains.
- Reject private/internal IP targets by default unless explicitly enabled for local development.
- Reject localhost, 127.0.0.0/8, 0.0.0.0/8, link-local, multicast, and cloud metadata IP ranges such as 169.254.169.254.
- Never concatenate SQL strings. Use SQLAlchemy ORM or parameterized queries only.
- Do not pass user input into shell commands.
- Normalize URLs and domains before storage and API calls.
- Set maximum input length, maximum JSON body size, and timeout limits.

- Escape or sanitize any user-controlled values displayed in HTML. React escapes by default, but avoid dangerouslySetInnerHTML.
- Store raw external API responses safely as JSON, not executable content.

10. Security Requirements

Threat	Required Defense
SQL Injection	Use ORM or parameterized queries, validate IDs as UUID, no raw query string interpolation.
DDoS / API abuse	Rate limit by IP, require CAPTCHA if public abuse appears, add Cloud Armor if using GCP, cache repeated lookups, request timeout limits.
API key leakage	Store keys in Secret Manager or environment variables, never expose in frontend, never log secrets.
SSRF	For URL/domain input, do not fetch arbitrary content directly except through approved APIs. Block private/internal IP resolution.
XSS	No raw HTML rendering, sanitize report text, use safe React rendering.
Command Injection	Do not run shell commands with user input. Use Python libraries instead.
Sensitive logging	Log request IDs and source names, not full secrets or sensitive tokens. Hash client IPs if stored.
Rate-limit exhaustion	Cache API results and show partial results when a source is unavailable.
Dependency vulnerabilities	Use pip-audit/npm audit, Dependabot, pinned dependencies, minimal Docker image.
CORS abuse	Allow only trusted frontend origin in production. Do not use wildcard CORS with credentials.

11. Risk Scoring System

Use a transparent weighted scoring model, not a black-box score. Keep it simple for MVP and document each rule.

Signal	Suggested Weight
VirusTotal malicious/suspicious detections	0-30 points
AbuseIPDB confidence and report volume	0-25 points
AlienVault OTX pulse matches and malware tags	0-20 points
URLScan malicious/suspicious verdicts or risky redirects	0-15 points
Shodan/Censys exposed risky services	0-10 points
IPinfo context, such as hosting provider or privacy/VPN flags	0-5 points
Age/cache confidence adjustments	-5 to +5 points

Severity bands: 0-24 Low, 25-49 Medium, 50-74 High, 75-100 Critical. Always show the evidence behind the score.

12. Dashboard Pages and UI Components

- Home / Analyze page: large input box, IOC type detector, analyze button, example inputs.
- Results page: risk score card, severity badge, AI summary, source status cards, evidence table.
- Source detail drawers: full normalized response for each tool without exposing raw secrets.
- History page: recent analyses, severity filter, source filter, search bar.
- Report export: JSON and optional PDF report for portfolio demo.
- Error states: partial results, rate-limited source, source unavailable, invalid input.

13. One-Month Build Plan

Week	Milestone	Deliverables
Week 1	Project foundation	Repo setup, FastAPI skeleton, React skeleton, Dockerfile, /health, input validation, IOC classifier, database schema.
Week 2	Core integrations	VirusTotal, AbuseIPDB, OTX, IPinfo connectors, normalized result schema, cache layer, basic risk scoring.
Week 3	Advanced integrations and dashboard	Shodan, Censys, URLScan connectors, async orchestration, dashboard cards, history page, source status handling.
Week 4	Security hardening and deployment	Rate limiting, SSRF protections, CORS, Cloud Run deployment, Secret Manager, README, screenshots, demo video, final polish.

14. Codex Execution Plan

Use Codex in small tasks. Do not ask it to build the entire app in one prompt. Give it this PDF, then provide one task at a time.

14.1 Initial Codex Prompt

You are building a secure cloud-hosted threat intelligence dashboard. Read the attached project plan fully before coding. Create the initial repo structure for a React TypeScript frontend and FastAPI Python backend. Follow the security requirements exactly: strict input validation, no raw SQL, no shell execution with user input, server-side API keys only, safe CORS, and clear separation between connectors, services, schemas, and routes. Do not implement all external APIs yet. First create the skeleton, Docker files, environment examples, /api/health, /api/analyze stub, IOC classifier, and tests for validation.

14.2 Suggested Codex Task Order

- Task 1: Create repo structure, backend skeleton, frontend skeleton, Docker files, and README draft.
- Task 2: Implement IOC classifier and validation tests.
- Task 3: Implement database models and safe ORM access.
- Task 4: Implement VirusTotal connector with mocked tests.
- Task 5: Implement AbuseIPDB, OTX, and IPinfo connectors.
- Task 6: Implement Shodan, Censys, and URLScan connectors.
- Task 7: Implement async orchestration and partial-result handling.
- Task 8: Implement risk scoring engine with unit tests.
- Task 9: Implement frontend dashboard components.
- Task 10: Implement security middleware: rate limiting, CORS, request size limits, timeouts, and secure headers.
- Task 11: Add deployment configs for Cloud Run and Vercel.
- Task 12: Add final README, screenshots, demo script, and resume bullet.

15. Testing Plan

Test Type	What to Test
Unit tests	IOC parsing, scoring rules, connector normalization, cache behavior.
Security tests	SQLi payloads, XSS strings, long inputs, invalid URLs, internal IPs, SSRF-like domains, malformed JSON.
Integration tests	Mock API responses for each connector and verify normalized output.
Frontend tests	Dashboard renders low/medium/high/critical states and source failures.
Deployment tests	Health endpoint, environment variables, secret access, CORS

	production origin.
Load sanity tests	Small controlled test to ensure rate limiting works. Do not perform real DDoS tests.

16. Security Checklist Before Launch

- All API keys are stored in Secret Manager or environment variables, never in frontend or GitHub.
- Backend CORS allows only the production frontend domain.
- Rate limiting is enabled on /api/analyze.
- Request body size is limited.
- All external API calls have timeouts.
- Invalid IOC inputs return 400.
- Private/internal IPs and metadata IPs are blocked.
- Database access uses ORM/parameterized queries only.
- No shell commands use user input.
- No dangerouslySetInnerHTML in frontend.
- Logs do not contain API keys or full raw secrets.
- Dependency scans run successfully.
- Cloud Run service account has minimum required permissions.
- README includes ethical use disclaimer and API setup instructions.

17. Suggested Repository Structure

```
cloud-threat-dashboard/
  backend/
    app/
      main.py
      core/config.py
      core/security.py
      api/routes/analyze.py
      api/routes/health.py
      schemas/ioc.py
      schemas/results.py
      services/classifier.py
      services/orchestrator.py
      services/risk_engine.py
      services/ai_summary.py
      connectors/virustotal.py
      connectors/abuseipdb.py
      connectors/otx.py
      connectors/shodan.py
      connectors/censys.py
      connectors/urlscan.py
      connectors/ipinfo.py
      db/models.py
      db/session.py
      tests/
    Dockerfile
    requirements.txt
    .env.example
  frontend/
    src/
```

components/
pages/
api/client.ts
types/results.ts
package.json
.env.example
docs/
screenshots/
architecture.md
README.md
docker-compose.yml

18. Ethical and Legal Guardrails

This project should only perform passive enrichment through approved third-party APIs. It should not attack, fuzz, exploit, brute force, or scan targets directly. Avoid building functionality that could be used to harm systems. Include a disclaimer that users must only analyze indicators they are authorized to investigate.

19. Resume and Portfolio Positioning

Resume bullet: Built a cloud-hosted threat intelligence dashboard using FastAPI, React, Google Cloud Run, and seven security APIs to analyze IPs, domains, URLs, and file hashes with automated risk scoring, caching, and AI-generated analyst summaries.

Portfolio title: Cloud Threat Intelligence Dashboard - Multi-Source IOC Enrichment and Risk Scoring Platform.

20. Definition of Done

- Public frontend URL works.
- Cloud Run backend URL works and is not exposing secrets.
- At least one successful demo exists for IP, domain, URL, and hash where supported.
- All seven connectors are implemented with graceful failure handling.
- Risk score is explainable and evidence-backed.
- Security checklist is complete.
- README explains setup, environment variables, architecture, screenshots, and ethical use.
- Demo video or GIF shows the dashboard workflow.